```
library(tidyverse)
library(lubridate)
library(openxlsx)
library(naniar)
library(ggplot2)
library(nnet)
library(caret)

# Set working directory of data
setwd("C:\\Users\\sethd\\OneDrive\\Desktop\\MS in Data Science Bellevue\\Classes\\DSC 630-T301
Predictive Analytics\\Milestone\\5")

# Import diabetes health data
dia_file <- read.csv("diabetes_012_health_indicators_BRFSS2015.csv")

# Change into singular dataframe for use
df <- dia_file %>%
  mutate(Diabetes_012 = as.factor(Diabetes_012))

df_target <- df %>% select(Diabetes_012)

df_features <- df %>% select(-Diabetes_012)

# Answering Data Questions ------------------------------------------------

# Confirming Diabetes_012 has three or more categories
summary.factor(df_target)

# Listing data types of data without target variable
str(df_features)

# List out factor data of dataframe without target variable
sapply(df_features, function(x) summary.factor(x))

# Data Visualizations -----------------------------------------------------

# Quick way to visually review dataframe to ensure no rows have missing values
# Using gg_miss_var function to review missing values of dataframe while
# showing results
gg_miss_var(df_features, show_pct = TRUE) +
  labs(x = "Feature Variable", y = "Missing Proportion")

# Show imbalance of target variable via bar chart
# Renaming target variables to values being measured
# Adding count of values
df_target_names <- df_target %>%
  group_by(Diabetes_012) %>%
  mutate(
    Diabetes_012 = case_when(
      Diabetes_012 == 0 ~ "No Diabetes/Only Pregnancy",
      Diabetes_012 == 1 ~ "Prediabetes",
      TRUE ~ "Has Diabetes"
    ),
    num_cnt = n()
  ) %>%
  ungroup() %>%
  unique() %>%
  rename("Participant Result" = Diabetes_012)

# Plotting bar chart
ggplot(df_target_names, aes(x = reorder(`Participant Result`, -num_cnt),
                            y = num_cnt, fill = `Participant Result`)) +
  geom_bar(stat = "identity") +
  xlab("Participant Result") +
```

```
  ylab("Number of Participants")

# Model Building ------------------------------------------------------------

# Model to be built is multinomial logistic regression model

###### Creating data partition to separate data into train/test
df_index <- createDataPartition(df$Diabetes_012, p = .80, list = FALSE)
train <- df[df_index, ]
test <- df[-df_index, ]


###### Setting the reference level to 0 for no diabetes
train$Diabetes_012 <- relevel(train$Diabetes_012, ref = "0")

###### Training the multinomial model
multinom_model <- multinom(Diabetes_012 ~ ., data = df)
###### Checking the model
summary(multinom_model)

####### Converting coefficients to odds by taking exponential of coefficients
odds_ratio = exp(coef(multinom_model))
odds_ratio

# Model Validation ----------------------------------------------------------

# Creating function to review p-values with TRUE showing p-values > 0.05
# and FALSE for p-values < 0.05
p_value_t_f <- function(x){
  zWald_modelo <- (summary(x)$coefficients /
                     summary(x)$standard.errors)
  a <- t(apply(zWald_modelo, 1, function(x) {x < qnorm(0.025, lower.tail = FALSE)} ))
  b <- t(apply(zWald_modelo, 1, function(x) {x > -qnorm(0.025, lower.tail = FALSE)} ))
  ifelse(a == TRUE & b == TRUE, TRUE, FALSE)
}

p_value_t_f(multinom_model)

###### Validating the model built on training data

# Predicting the values for train dataset
train$ClassPredicted <- predict(multinom_model, newdata = train, "class")
# Building classification table
tab <- table(train$Class, train$ClassPredicted)
# Calculating accuracy - sum of diagonal elements divided by total obs
round((sum(diag(tab))/sum(tab))*100,2)

### Predicting the class on test data
# Predicting the class for test dataset
test$ClassPredicted <- predict(multinom_model, newdata = test, "class")
# Building classification table
tab <- table(test$Class, test$ClassPredicted)
tab
```